GitTrend Radar (Git Search) v0.1.3

Guia Oficial do Aluno | Metodo CRIAR | Tauri 2 (Rust) + React 19 + Ollama Local

Nome do Projeto: GitTrend Radar (Git Search)	**Versao:** 0.1.3 (Outubro 2026)
Tecnologias: Tauri 2, Rust 1.85+, React 19, SQLite	**Tamanho do Pacote:** 4,39 MiB (Instalador NSIS)
IA Local Embarcada: Ollama (Qwen 3 / SmolLM2)	**Publico Alvo:** Alunos da Formacao Vibe Coder

1. O que e o Git Search?

O GitTrend Radar e uma ferramenta para programadores e agentes autonomos que consolida o que ha de mais relevante no ecossistema open source. Em vez de navegar manualmente no GitHub e perder horas lendo documentacoes extensas, o aplicativo analisa os repositorios em alta, os modelos de IA recem-publicados e as skills de agentes, gerando sinteses precisas em portugues sem gastar nenhum centavo com tokens em nuvem.

2. O Metodo CRIAR na Construcao do Software

C	**Contexto:** Milhares de novidades sao lancadas diariamente no GitHub e Hugging Face. A sobrecarga de informacao impede o desenvolvedor de focar no que realmente importa.
R	**Realidade:** Solucoes baseadas em Electron gastam muita memoria RAM (400MB+). Com Tauri 2 e Rust, o app consome menos de 60MB e roda com inicializacao instantanea.
I	**Imersao:** O codigo integra chamadas nativas assincronas em Rust com componentes visuais reativos no React 19, utilizando SQLite em disco para cache de 7 dias.
A	**Aplicacao:** O aluno aprende a criar fluxos reais de download com Git LFS, orquestrar prompts locais no Ollama e estruturar software profissional para distribuicao.
R	**Resultado:** Um binario ultraleve (.exe) de 4,39 MB pronto para uso diario no computador do aluno, acelerando a descoberta e estudo de novas ferramentas.

3. Guia de Instalacao e Execucao (Passo a Passo)

Opcao A: Instalacao Rapida via Executavel Pronto

Para utilizar imediatamente sem necessidade de compilar:
1. Navegue ate a pasta do instalador: frontend/src-tauri/target/release/bundle/nsis/
2. Execute o instalador GitTrend Radar_0.1.3_x64-setup.exe com dois cliques.
3. Conclua o assistente e acesse o aplicativo pelo Menu Iniciar do Windows.

Opcao B: Execucao em Modo de Desenvolvimento (Para Programar)

Para inspecionar o codigo fonte, alterar telas e aprender a arquitetura:
1. Pre-requisitos do Ambiente: Node.js v20+, Rust 1.85+ (via rustup) e Git com Git LFS.
2. Comandos no Terminal PowerShell:

```
# 1. Acesse o diretorio frontend
cd "E:\REPOSITORIOS\GIT_SEARCH\frontend"

# 2. Instale as dependencias do Node
npm install

# 3. Inicie o ambiente Tauri 2 com Hot-Reload
npx tauri dev
```

4. Arquitetura do Sistema e Fluxo de Dados

A aplicacao e dividida em tres camadas fundamentais, garantindo alto desacoplamento e maxima performance:

Camada	Tecnologia	Funcao no Projeto
Interface (Frontend)	React 19 + Vite	Renderizacao reativa dos cards de repositorios, modelos e skills. Zero manipulacao direta de DOM; estado declarativo.
Ponte IPC (Bridge)	Tauri 2 Runtime	Comunicacao segura via comandos tipados (tauri::command) e despacho assincrono de eventos de progresso.
Backend Nativo	Rust (Tokio + SQLite)	Scraping veloz do GitHub, requisicoes ao Hugging Face, clonagem em disco e persistencia relacional.
IA Local	Ollama API	Traducao e resumo conciso em portugues (maximo 55 palavras) rodando nos modelos locais da maquina.

5. Mapa de Arquivos: Onde Estudar e Modificar

Como aluno da Bianchini Academy, seu foco deve ser entender a interacao entre os arquivos chave do projeto:

Arquivo	Linguagem	O que voce aprende analisando este arquivo
`src-tauri/src/lib.rs`	Rust	Como registrar comandos Tauri, inicializar o banco de dados e gerenciar o ciclo de vida do app desktop.
`src-tauri/src/github.rs`	Rust	Como coletar informacoes publicas do GitHub, fazer parsing de HTML com scraper e consumir APIs com reqwest.
`src-tauri/src/catalog.rs`	Rust	Integracao com o catalogo do Hugging Face e repositorios abertos de skills de inteligencia artificial.
`src-tauri/src/summaries.rs`	Rust	Criacao de fila assincrona com controle de concorrencia para enviar prompts estruturados ao Ollama.
`src/components/ContentCard.jsx`	React 19	Construcao de cards expansivos modernos com estados de progresso, botoes e suporte a acessibilidade (ARIA).
`src/components/cardDetails.js`	JavaScript	Logica pura de formatacao de fatos, descricao, veredito e listas de compatibilidade.

6. Desafios Praticos para o Aluno (Mao na Massa)

A melhor forma de consolidar a formacao de Vibe Coder e implementar novas funcionalidades sobre a base existente:

Desafio 1 (Nivel Basico): Botao de Copiar URL
No componente ContentCard.jsx, adicione um icone com funcao para copiar o link oficial do repositorio ou modelo para a area de transferencia do Windows, exibindo um feedback visual rapido de sucesso.

Desafio 2 (Nivel Intermediario): Ajuste Fino do Prompt da IA
No arquivo src-tauri/src/summaries.rs, localize o prompt enviado ao Ollama. Experimente alterar a instrucao para exigir que a IA inclua sempre a stack tecnica principal no final do resumo.

Desafio 3 (Nivel Avancado): Comando de Exportacao
Implemente um novo comando Tauri em Rust chamado export_library_markdown que consulte a tabela de favoritos do SQLite e gere um arquivo favoritos.md formatado na pasta Downloads do usuario.

7. Checklist de Conclusao e Proximos Passos

[ ] Aplicativo instalado e fixado na barra de tarefas.
[ ] Primeiro clone de repositorio realizado com sucesso para E:\REPOSITORIOS.
[ ] Ollama testado localmente com geracao de resumos em portugues.
[ ] Codigo fonte aberto na IDE e primeiro desafio concluido.